Java did NOT introduce Scala in Java 1.8.
Scala is a **separate language (2004)**.
Java 1.8 borrowed / was inspired by ideas popularized by Scala (and other functional languages).

Now let's do this **cleanly, in tables**, exactly as you asked.

---

1 Java vs Scala — Language Overview

Parameter	Java	Scala
First Released	1995	2004
Developed By	Sun Microsystems (now Oracle)	Martin Odersky
Runs On	JVM	JVM
Paradigm	Object-Oriented (Java 8+: mixed)	Object-Oriented + Functional
Backward Compatibility	Very strong	Moderate
Syntax Style	Verbose	Concise
Learning Curve	Easier	Steeper
Compilation Output	.class files	.class files
Interoperability	Native	100% interoperable with Java

---

2 Core Language Features Comparison

Feature	Java	Scala	Similar / Different
Object-Oriented	Yes	Yes	✅ Similar
Functional Programming	Limited (Java 8+)	First-class	❌ Different
Lambdas	Java 8+	From day one	❌
Type Inference	Very limited	Strong	❌
Immutability	Optional	Preferred	❌

Null Safety	No	Option type	✕
Multiple Inheritance	No (interfaces only)	Via traits	✕
Pattern Matching	No (switch limited)	Yes (powerful)	✕
Checked Exceptions	Yes	No	✕

---

3 Syntax Comparison Example

Lambda

Java	**Scala**
(a, b) -> a + b	(a, b) => a + b

Variable Declaration

Java	**Scala**
int x = 10;	val x = 10

---

4 Functional Programming Support

Concept	**Java**	**Scala**
Functions as Values	Partial	Yes
Higher-Order Functions	Limited	Yes
Closures	Yes (Java 8+)	Yes
Lazy Evaluation	No	Yes
Immutability by Default	No	Yes

---

5 Collections Framework

Aspect	Java	Scala
Mutable Collections	Yes	Yes
Immutable Collections	Limited	Rich & Default
Streams / Operations	Streams API	Built-in
map / filter / reduce	Via Streams	Native

---

6 Concurrency Model

Feature	Java	Scala
Threads	Yes	Yes
Executors	Yes	Yes
Futures	Java 8+	Earlier & richer
Actor Model	No	Yes (Akka)
Async Style	Verbose	Concise

---

7 Why Java 1.8 Added Scala-like Features

Feature Added in Java 8	Inspired by Scala	Why
Lambda Expressions	✅ Yes	Reduce verbosity
Functional Interfaces	✅ Yes	Enable FP
Stream API	✅ Yes	Declarative data processing
Method References	✅ Yes	Cleaner syntax
Default Methods	✅ Yes	Interface evolution

 Reason:
Java was losing ground to **Scala, Groovy, Kotlin** due to verbosity and weak FP support.

---

8 Core Concepts Introduced in Java 1.8 (Scala Influence)

Concept	Explanation
Lambda Expressions	Functions without class boilerplate
Functional Interfaces	Single abstract method interfaces
Streams	Functional data pipelines
Method References	Shorter lambda syntax
Default Methods	Behavior inside interfaces

---

9 Philosophy Difference

Aspect	Java	Scala
Design Goal	Stability & simplicity	Power & expressiveness
Code Style	Explicit	Expressive
Enterprise Focus	Very strong	Strong but niche
Ecosystem	Huge	Smaller but advanced

□ Summary (One-Line)

Java	Scala
Safer, verbose, enterprise-friendly	Powerful, concise, FP-heavy
Java 8 borrowed ideas	Scala influenced Java evolution
Easier to maintain	Faster to write complex logic

 Final Takeaway

- **Scala did not come from Java**
- **Java 8 came closer to Scala**
- Java = **stability**
- Scala = **expressiveness**

If you want, I can also:

- Compare **Java vs Scala bytecode**
- Explain **why Kotlin beat Scala**
- Show **same program in Java 7 vs Java 8 vs Scala**